

Job Scheduler System Design - Complete Interview Guide

Comprehensive guide with diagrams, tables, code, and explanations

Print Settings: Landscape mode, monospace font (Courier New/Consolas 9-10pt), narrow margins

SECTION 1: REQUIREMENTS & CAPACITY ESTIMATION

1.1 Functional Requirements

- ✓ Schedule one-time jobs (run at specific time)
- ✓ Schedule recurring jobs (cron-style, e.g., every 5 mins)
- ✓ Support job priorities (high/medium/low)
- ✓ Track job status (pending/running/succeeded/failed)
- ✓ Retry failed jobs with configurable policy
- ✓ Cancel/pause/resume scheduled jobs
- ✓ Support job dependencies (Job B runs after Job A)
- x Real-time streaming pipelines (out of scope)
- x User-facing workflow orchestration like Airflow (out of scope)

How to Present: "Before I jump into the design, let me clarify the requirements. I'll focus on:

- Users can submit jobs to run at a specific time, or on a cron schedule like 'every 5 minutes'
- Jobs can have priorities so urgent jobs don't get starved behind low-priority work
- We need full status tracking - you should be able to query a job and know if it's pending, running, succeeded, or failed
- Retries are important - if a job fails due to a transient error, we should retry automatically
- Job dependencies: some workflows require Job B only runs after Job A completes

I'll scope out complex workflow DAGs like Airflow and real-time stream processing."

1.2 Non-Functional Requirements

Scale:	10 million jobs/day (~116 jobs/sec avg, ~1,000 jobs/sec peak)
Latency:	Job starts within 1 second of scheduled time
Availability:	99.99% (no job is silently dropped)
Consistency:	Every job executes AT LEAST ONCE (prefer over at-most-once)
Throughput:	1,000 concurrent running jobs
Job Sizes:	Short (ms) to long-running (hours)

How to Explain: "From a non-functional perspective:

- **Scale:** Roughly 10 million jobs per day. That's ~116 per second on average, maybe 1,000 during peak hours like batch report generation at midnight.
- **Latency:** A job scheduled for 10:00:00 AM should start by 10:00:01. One-second accuracy is acceptable.
- **Availability:** 99.99% means we cannot silently drop jobs. If a scheduler node dies, its jobs must be picked up by another node.
- **Consistency:** We prefer AT-LEAST-ONCE execution over AT-MOST-ONCE. It's safer to run a job twice (with idempotency at the job level) than to drop it entirely."

1.3 Capacity Estimation

JOB PER DAY:	10 million
JOB PER SECOND:	~116 avg, ~1,000 peak
CONCURRENT JOBS:	1,000
JOB METADATA SIZE:	~2 KB per job
STORAGE (1 year):	10M * 365 * 2KB = ~7 TB (metadata + history)
WORKER NODES:	1,000 workers (each handles 1 long job or 100 short jobs)

How to Walk Through: "Let me estimate capacity:

Job rate: 10 million per day / 86,400 seconds $\approx$ 116 jobs/sec average. But jobs don't arrive uniformly - nightly batch processing means we might see 1,000 jobs/sec around midnight.

Storage: Each job record is roughly 2KB (metadata: id, schedule, payload, status, retry policy, timestamps). 10M jobs/day $\times$ 365 days $\times$ 2KB = ~7TB per year. We'd archive old job history to cold storage after 90 days.

Workers: Assume 1,000 concurrent running jobs. A worker running a lightweight job (DB backup) handles it in 100ms. A heavy job (ML training) takes hours. We need a pool that can handle mixed workloads."

CROSS-QUESTIONS & ANSWERS:

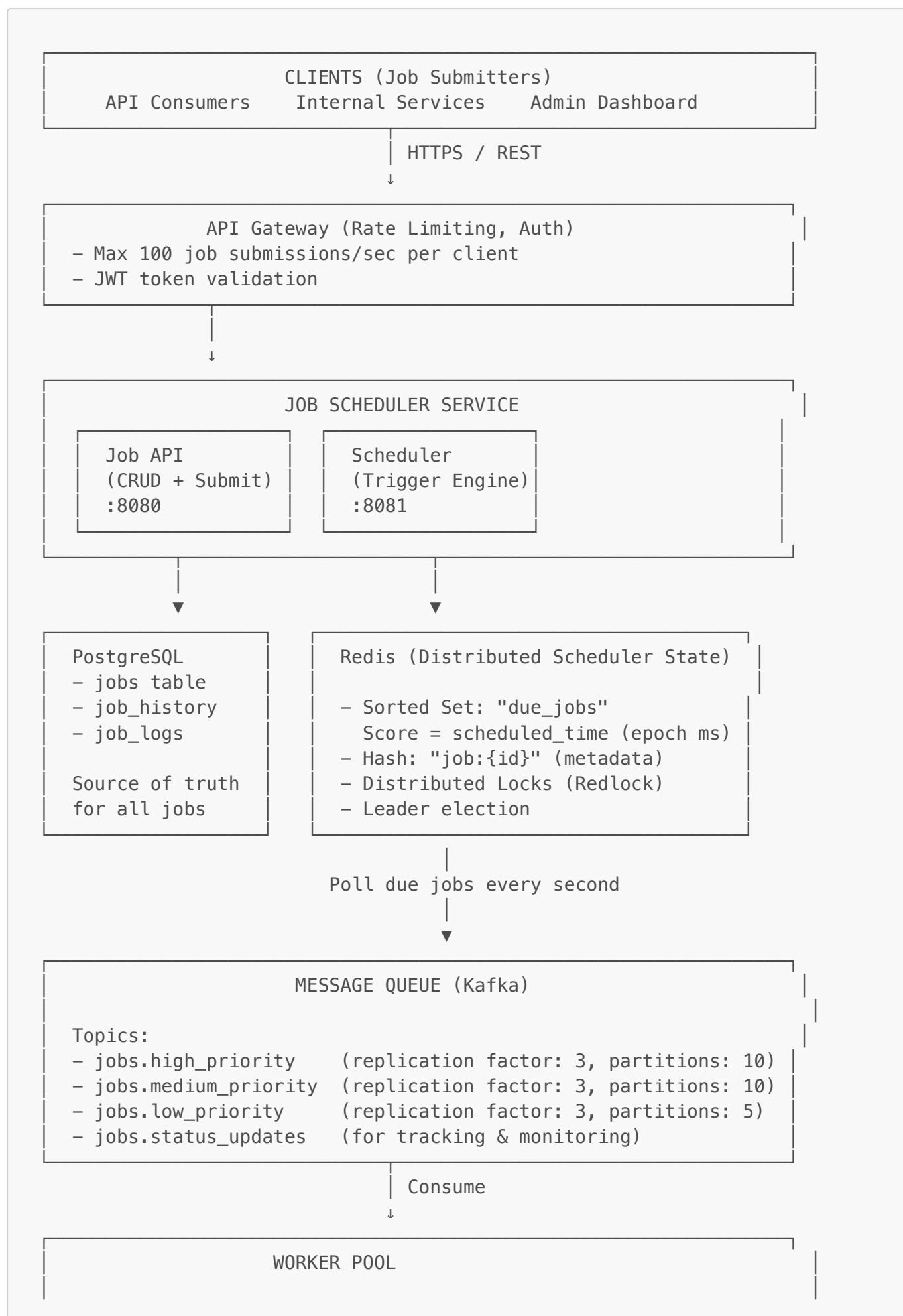
Q1: Why AT-LEAST-ONCE instead of EXACTLY-ONCE? "EXACTLY-ONCE requires distributed transactions which are extremely expensive to implement correctly. AT-LEAST-ONCE is achievable with simpler mechanisms: once a worker claims a job, if the worker dies, another worker retries it. The solution is to make jobs **idempotent** - so running them twice has the same result as running once. Example: an 'update user balance' job uses an idempotency key to check if already applied."

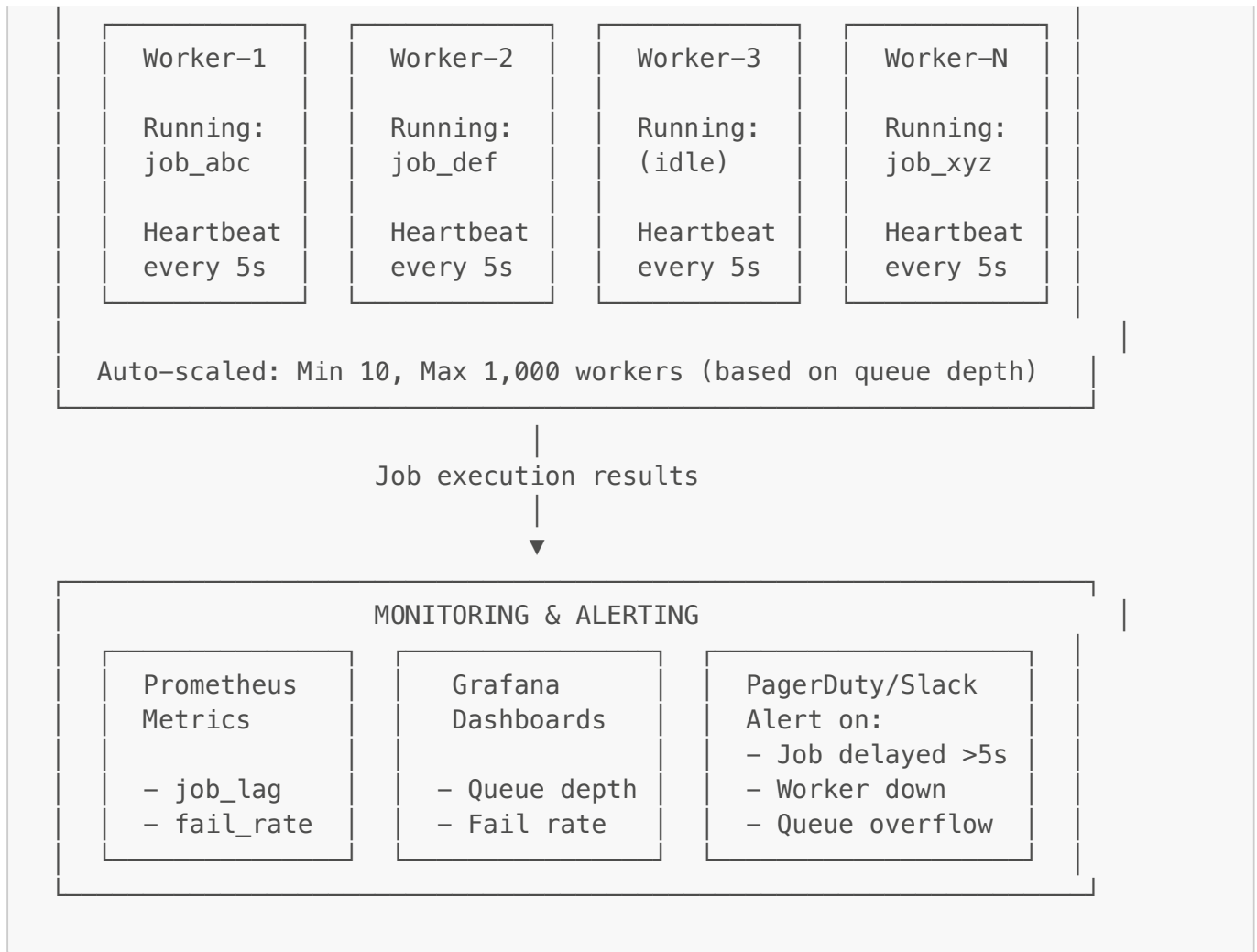
Q2: What if 10 million jobs all have the same schedule time? "Great edge case. If every job is 'run at midnight', we'd have 10M jobs due simultaneously. We handle this with:

1. **Sharding the job queue by time bucket:** Each worker owns a time range
2. **Priority queues:** Higher priority jobs go first
3. **Rate limiting execution:** Throttle to prevent thundering herd
4. **Horizontal scaling:** Spin up extra workers before midnight using auto-scaling triggers"

SECTION 2: HIGH-LEVEL ARCHITECTURE

2.1 System Architecture Diagram





2.2 How to Draw & Explain

Drawing Strategy (Step-by-Step):

"Let me draw this from top to bottom, starting with job submission and ending with execution:

Step 1 - Clients: 'Job submitters can be anything: a service that wants to send an email at midnight, a billing system that charges users monthly, a data pipeline that runs every hour. They submit jobs via REST API.'

Step 2 - API Gateway: 'Requests go through an API Gateway for:

- JWT authentication: only authorized services can submit jobs
- Rate limiting: prevent abuse - max 100 submissions/sec per client
- Request validation before hitting our service'

Step 3 - Scheduler Service (two components): 'The core of the system has two parts:

- Job API: handles CRUD - create, update, cancel, query job status
- Scheduler Engine: runs in the background, every second polls for jobs that are due to run

These are separate because the API needs to be fast and responsive. The scheduler runs on a loop and should not be mixed with user-facing requests.'

Step 4 - Storage (dual-write): 'We maintain two stores:

- PostgreSQL: the permanent source of truth for all jobs, history, logs
- Redis Sorted Set: an in-memory index of jobs sorted by their scheduled time

Why Redis for scheduling? We need to efficiently query "which jobs are due RIGHT NOW?" PostgreSQL with timestamps works, but under high load, a Redis sorted set gives us $O(\log N)$ range query with millisecond latency.'

Step 5 - Kafka: 'The scheduler doesn't execute jobs itself. When a job is due, the scheduler pushes it to Kafka. Why?

- Decouples scheduling from execution
- Workers can go down without losing the job (Kafka retains it)
- We can have different queues for different priorities'

Step 6 - Worker Pool: 'Workers pull jobs from Kafka and execute them. Each worker:

- Marks the job as "running" in the database
- Sends heartbeats every 5 seconds
- If a worker dies, no heartbeat is received and the job gets retried
- Reports success/failure back to update job status"

2.3 Key Design Decisions

Component	Decision	Reasoning
Redis Sorted Set lookup	In-memory time-indexed queue	$O(\log N)$ due-job
Kafka Topics	Separate by priority	High priority not blocked by low
Worker Heartbeat reliably	5-second heartbeat + 30s timeout	Detect dead workers
PostgreSQL audit history	Source of truth for job state	ACID guarantees,
Distributed Lock scheduling	Redis Redlock for leader election	Prevent duplicate

CROSS-QUESTIONS & ANSWERS:

Q1: Why use Redis sorted set for scheduling? Why not just query PostgreSQL directly? "Great question. Let me compare both approaches:

PostgreSQL Polling:

```
SELECT * FROM jobs
WHERE status = 'pending'
      AND scheduled_at <= NOW()
ORDER BY scheduled_at ASC
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

This works but at 1,000 jobs/sec with multiple schedulers, we'd hit PostgreSQL with thousands of queries per second. Under load, this creates lock contention and database hotspots.

Redis Sorted Set:

```
ZRANGEBYSCORE due_jobs 0 {current_epoch_ms} LIMIT 0 100
```

Redis processes this in microseconds from memory. No lock contention. No I/O.

The Trade-off: Redis can lose data on restart (if AOF is not enabled), while PostgreSQL is always durable. The solution: PostgreSQL is the source of truth. Redis is a fast index that gets rebuilt from PostgreSQL on startup. So we get the durability of Postgres with the speed of Redis."

Q2: How do you prevent a job from being executed twice (double dispatch)? "This is the most critical race condition in a job scheduler. Two schedulers could both poll Redis at the same millisecond and dispatch the same job.

Solution 1 - Leader Election: Only ONE scheduler node polls Redis and dispatches jobs. Others are standby. If the leader dies, a new leader is elected. We use Redis Redlock for this.

Solution 2 - Atomic Move with Redis: When dispatcher claims a job, it atomically removes it from 'due_jobs' sorted set and puts it in 'in_flight' set using a Lua script (executed atomically in Redis):

```
local job = redis.call('ZRANGEBYSCORE', 'due_jobs', 0, ARGV[1], 'LIMIT', 0,
1)
if job[1] then
    redis.call('ZREM', 'due_jobs', job[1])
    redis.call('HSET', 'in_flight', job[1], ARGV[2]) -- ARGV[2] =
worker_id
end
return job
```

Since Lua scripts are atomic in Redis, no two dispatchers can claim the same job.

Solution 3 - Database-level optimistic lock: When updating status from PENDING → RUNNING, use:

```
UPDATE jobs SET status='RUNNING', worker_id=?
WHERE id=? AND status='PENDING';
```

```
-- Affects 0 rows if already claimed by another worker
```

Only the worker that sees rowcount=1 proceeds."

Q3: What if a worker crashes while running a job? "This is the **stale job / zombie job** problem. Here's the full solution:

Heartbeat Mechanism:

- Every running worker sends a heartbeat every 5 seconds:

```
HSET worker:heartbeats {worker_id} {current_timestamp}
```

- A separate Reaper process checks for workers with stale heartbeats (>30 seconds old)

Reaper Process:

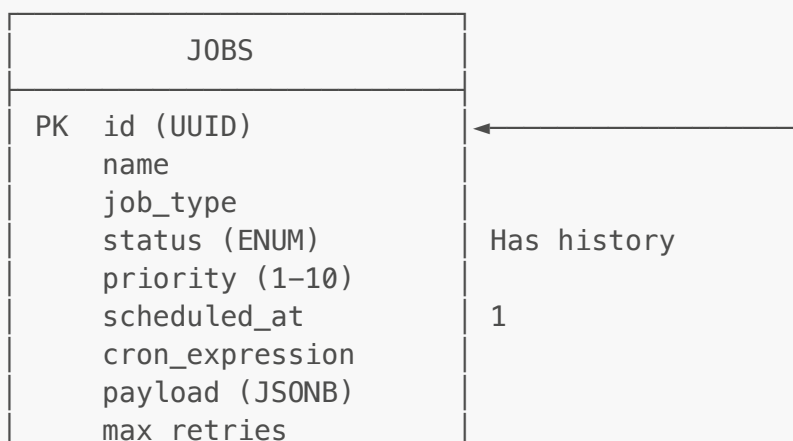
Every 30 seconds:

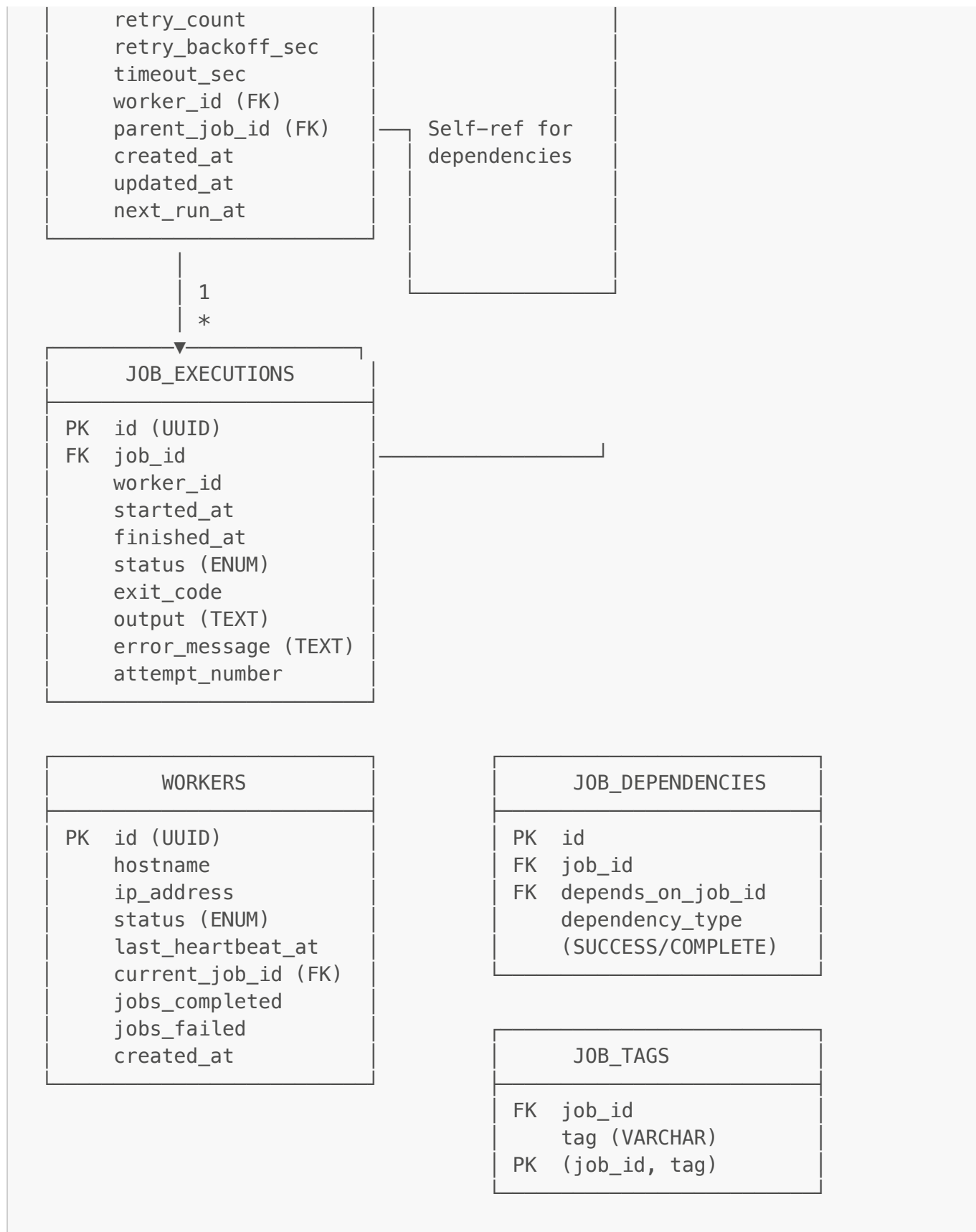
1. Scan all jobs with status='RUNNING'
2. For each running job, check if worker is alive
3. If worker heartbeat is >30 seconds old:
 - Mark worker as DEAD
 - Move job back to PENDING
 - Increment job's retry_count
 - If retry_count < max_retries, re-schedule with backoff
 - If retry_count >= max_retries, mark job as FAILED

Result: A crashed worker's jobs are automatically recovered within 30 seconds. No manual intervention needed."

SECTION 3: DATABASE DESIGN

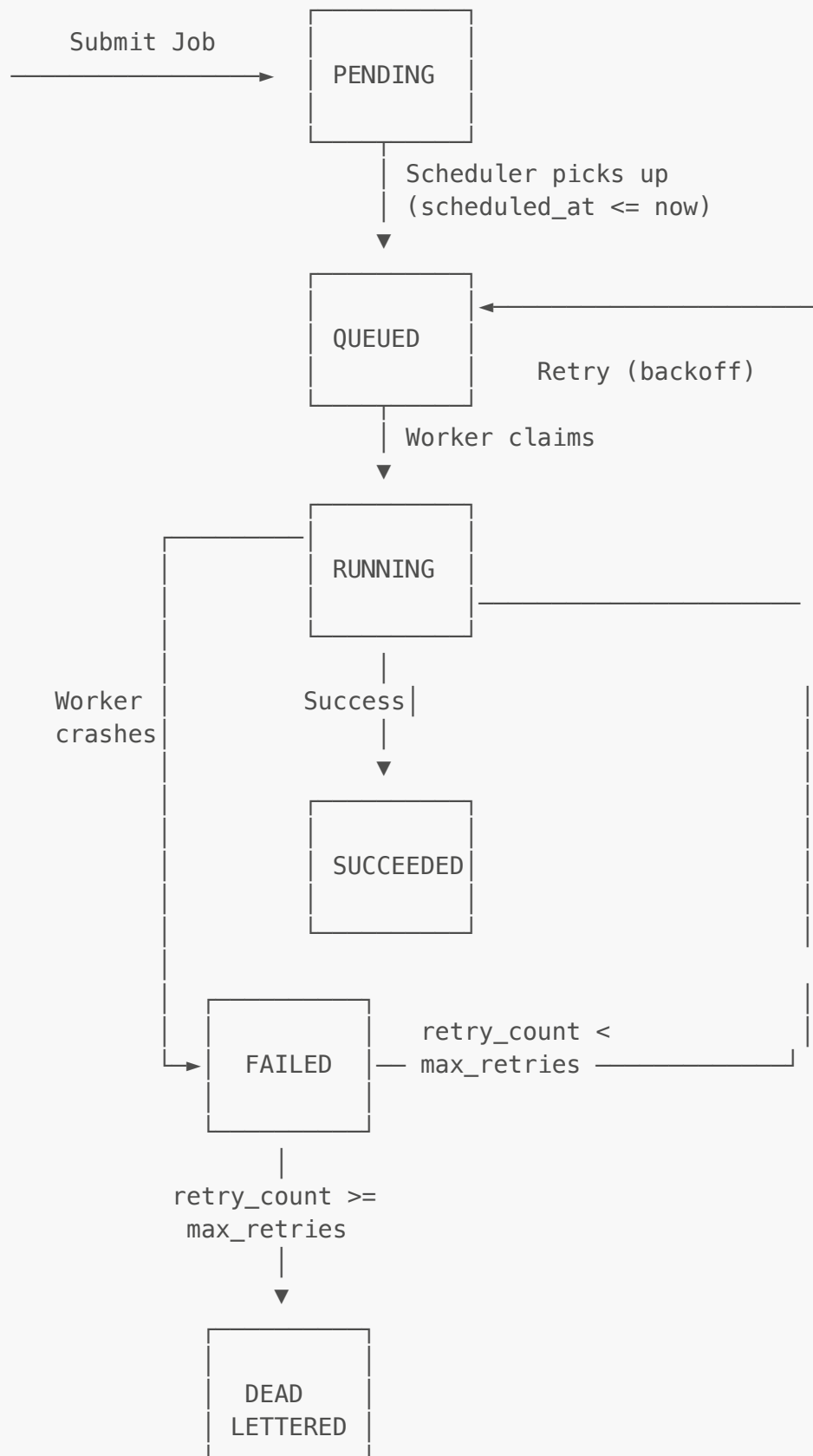
3.1 Entity Relationship Diagram





3.2 Job Status State Machine

Job Status Transitions



Also:

PENDING — Cancel —> CANCELLED
 RUNNING — Timeout —> FAILED
 PENDING — Pause —> PAUSED — Resume —> PENDING

3.3 Table Structure & Index Strategy

TABLE: jobs

(source of truth, ~10M rows/month)

Column

Type

Constraint / Note

id

UUID

PK, gen_random_uuid()

name

VARCHAR(255)

NOT NULL

job_type

VARCHAR(100)

NOT NULL e.g.

'email','report','export' |

status

VARCHAR(20)

NOT NULL DEFAULT 'PENDING'

CHECK IN (PENDING, QUEUED, RUNNING,

SUCCEEDED, FAILED, CANCELLED,

PAUSED, DEAD_LETTERED)

priority

SMALLINT

10=lowest) | NOT NULL DEFAULT 5 (1=highest,

scheduled_at

TIMESTAMPTZ

NOT NULL

cron_expression

VARCHAR(100)

NULLABLE (null = one-time job)

payload

JSONB

NULLABLE (job-specific params)

max_retries

SMALLINT

NOT NULL DEFAULT 3

retry_count

SMALLINT

NOT NULL DEFAULT 0

retry_backoff_sec

INT

NOT NULL DEFAULT 60

timeout_sec

INT

NOT NULL DEFAULT 3600

worker_id

UUID

FK → workers.id NULLABLE

parent_job_id

UUID

FK → jobs.id NULLABLE (self-ref)

next_run_at

TIMESTAMPTZ

NULLABLE (computed for recurring jobs)

created_at

TIMESTAMPTZ

DEFAULT now()

updated_at

TIMESTAMPTZ

DEFAULT now()

INDEXES (critical for scheduler performance):

```
idx_jobs_scheduled_status  ON (scheduled_at, status)
                             WHERE status IN ('PENDING','PAUSED')
                             → Partial index, tiny footprint, fast poll

idx_jobs_status_priority    ON (status, priority)
                             WHERE status = 'PENDING'
                             → Used by priority-ordered dispatch
```

TABLE: job_executions (audit log, append-only)

id	UUID	PK
job_id	UUID	FK → jobs.id ON DELETE CASCADE
worker_id	UUID	FK → workers.id NULLABLE
attempt_number	SMALLINT	NOT NULL DEFAULT 1
status	VARCHAR(20)	NOT NULL
started_at	TIMESTAMPTZ	NOT NULL DEFAULT now()
finished_at	TIMESTAMPTZ	NULLABLE
exit_code	INT	NULLABLE
output	TEXT	NULLABLE (stdout/result)
error_message	TEXT	NULLABLE

INDEX: idx_executions_job_id ON (job_id, started_at DESC)

→ Fast lookup of all attempts for a job, newest first

TABLE: workers (live registry of worker nodes)

id	UUID	PK
hostname	VARCHAR(255)	NOT NULL
ip_address	INET	NOT NULL
status	VARCHAR(20)	NOT NULL DEFAULT 'IDLE'
		(IDLE BUSY DRAINING DEAD)
last_heartbeat	TIMESTAMPTZ	NOT NULL DEFAULT now()
current_job_id	UUID	FK → jobs.id NULLABLE
jobs_completed	BIGINT	DEFAULT 0
jobs_failed	BIGINT	DEFAULT 0
created_at	TIMESTAMPTZ	DEFAULT now()

INDEX: idx_workers_heartbeat ON (last_heartbeat)

WHERE status = 'BUSY'

→ Reaper uses this to find stale workers in one fast scan

WHY JSONB PAYLOAD (not separate columns):

Email job: { "to": "...", "subject": "...", "template_id": "..." }

Report job: { "report_type": "...", "date_range": "...", "recipients":
[] }

Export job: { "table_name": "...", "filters": {}, "s3_path": "..." }

→ 50+ columns would be mostly NULL. JSONB keeps one clean table.

→ Can still index: CREATE INDEX ON jobs ((payload->>'report_type'))
WHERE job_type = 'report';

CROSS-QUESTIONS & ANSWERS:

Q1: Why store cron_expression in the jobs table? "For recurring jobs, the cron expression IS part of the job definition. When a job with cron='0 * * * *' (every hour) succeeds, the scheduler automatically computes next_run_at using the cron expression and creates the next execution. The jobs table effectively acts as a job template for recurring jobs. A separate 'schedules' table is also valid, but keeping it in one table simplifies queries."

Q2: Why partial indexes on the jobs table? "The scheduler only ever queries PENDING or PAUSED jobs. A partial index `WHERE status IN ('PENDING', 'PAUSED')` means the index only contains the ~1% of rows that are actively being scheduled, not the 99% that are already SUCCEEDED or FAILED. This keeps the index tiny, so the scheduler's poll query hits L1 cache in most cases. At 10M rows/month without partials, the index is tens of millions of rows. With partials, it's only tens of thousands."

SECTION 4: SCHEDULER TRIGGER ENGINE

4.1 How the Scheduler Polls & Dispatches

SCHEDULER TRIGGER LOOP
(runs every 1 second)

Step 1: Acquire Leader Lock (Redlock)

Multiple scheduler nodes compete for leader lock in Redis:
`SET scheduler:leader {node_id} NX PX 5000` (5 second TTL)

Only the winner proceeds. Others wait.
Winner renews lock every 2 seconds.

Step 2: Query Due Jobs from Redis Sorted Set

`ZRANGEBYSCORE due_jobs 0 {now_epoch_ms} LIMIT 0 100`

Redis Sorted Set: "due_jobs"		
Score	Member (job_id)	
1691000000	job_aaa	← Due now
1691000001	job_bbb	← Due now
1691000002	job_ccc	← Due now
1691005000	job_ddd	(5 min future)
1691010000	job_eee	(10 min future)

Step 3: Atomically Claim Job (Lua Script)

For each due job:

- Lua script removes from due_jobs
- Adds to in_flight set with TTL
- Atomic = cannot be claimed twice

Step 4: Update Job Status in PostgreSQL

```
UPDATE jobs SET status='QUEUED', updated_at=now()
WHERE id=? AND status='PENDING'
RETURNING id;
```

If 0 rows affected → job was already claimed, skip.

Step 5: Publish to Kafka by Priority

```
IF priority <= 3 → Topic: jobs.high_priority
IF priority <= 7 → Topic: jobs.medium_priority
ELSE              → Topic: jobs.low_priority
```

Step 6: Schedule Next Occurrence for Recurring Jobs

If job has cron_expression:

- Calculate next_run_at using cron parser
- Create new job entry with status='PENDING'
- Add to Redis sorted set with new score

4.2 Redis Data Structures Used

REDIS KEY DESIGN

due_jobs (Sorted Set)

Key: due_jobs
Score: epoch_ms (scheduled time)
Member: job_id
Query: ZRANGEBYSCORE due_jobs 0 {now}

in_flight (Hash)

Key: in_flight:{job_id}
Field: worker_id, claimed_at, timeout_at
TTL: job timeout + buffer

scheduler:leader (String)

Key: scheduler:leader
Value: {node_id}
TTL: 5 seconds (renewed every 2s)

worker:heartbeat (Hash)

```

Key: worker:heartbeat
Field: {worker_id}
Value: {timestamp}
Updated: every 5 seconds by each worker

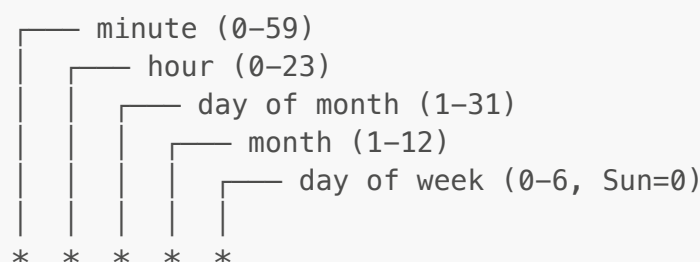
job:status:{job_id} (String)

Fast status lookup cache (TTL: 60 seconds)

```

4.3 Cron Expression Parsing

CRON FORMAT:



EXAMPLES:

```

0 * * * * → Every hour at minute 0
*/5 * * * * → Every 5 minutes
0 0 * * * → Every day at midnight
0 0 * * 1 → Every Monday at midnight
0 9 * * 1-5 → Weekdays at 9 AM (Mon-Fri)
0 0 1 * * → First day of every month

```

IMPLEMENTATION: Use a library like Quartz (Java) or cron-parser (Node.js).
Next run calculation:

```

nextRun = CronExpression.parse("0 * * * *")
               .nextTimeAfter(lastRunTime);

```

CROSS-QUESTIONS & ANSWERS:

Q1: What if the scheduler node (leader) itself crashes? "This is the exact reason we use Redlock with a short TTL:

- Leader holds the lock with TTL = 5 seconds
- Leader renews every 2 seconds: it'll renew 2+ times before TTL expires
- If the leader crashes, it stops renewing
- After 5 seconds, the lock expires in Redis
- Another scheduler node wins the next lock acquisition
- The new leader reads Redis and PostgreSQL to find any jobs that were queued but not yet dispatched (status=QUEUED but no worker claimed them)
- Recovery time: max 5 seconds

Why 5 seconds and not 1 second? Short enough to recover fast, long enough to tolerate network hiccups that temporarily prevent the leader from renewing."

Q2: What about time zones for scheduled jobs? "All timestamps in the database are stored in UTC. Clients specify jobs with a timezone offset that we convert to UTC at submission time.

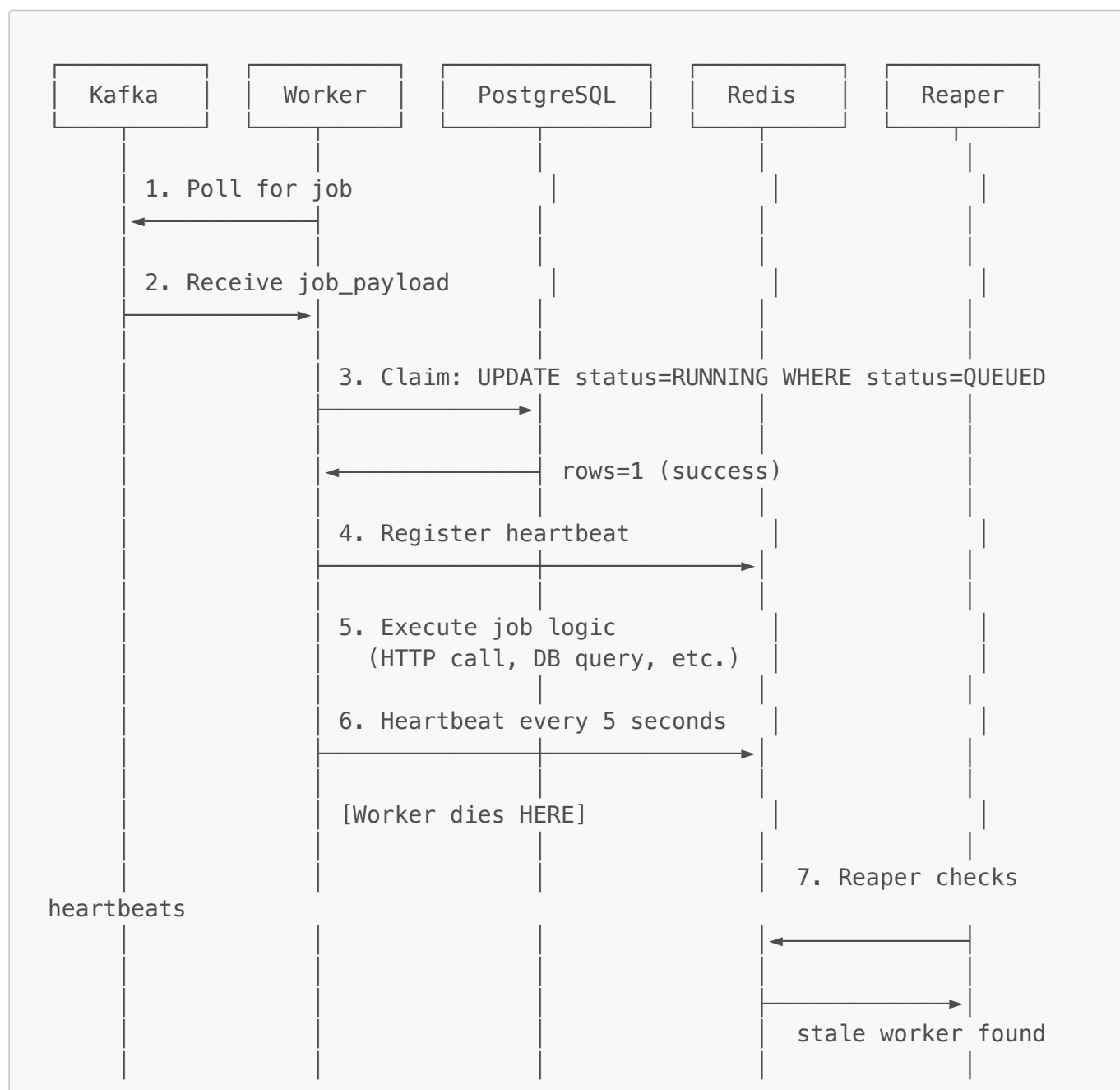
For cron jobs, the cron expression is evaluated in the timezone the client specifies:

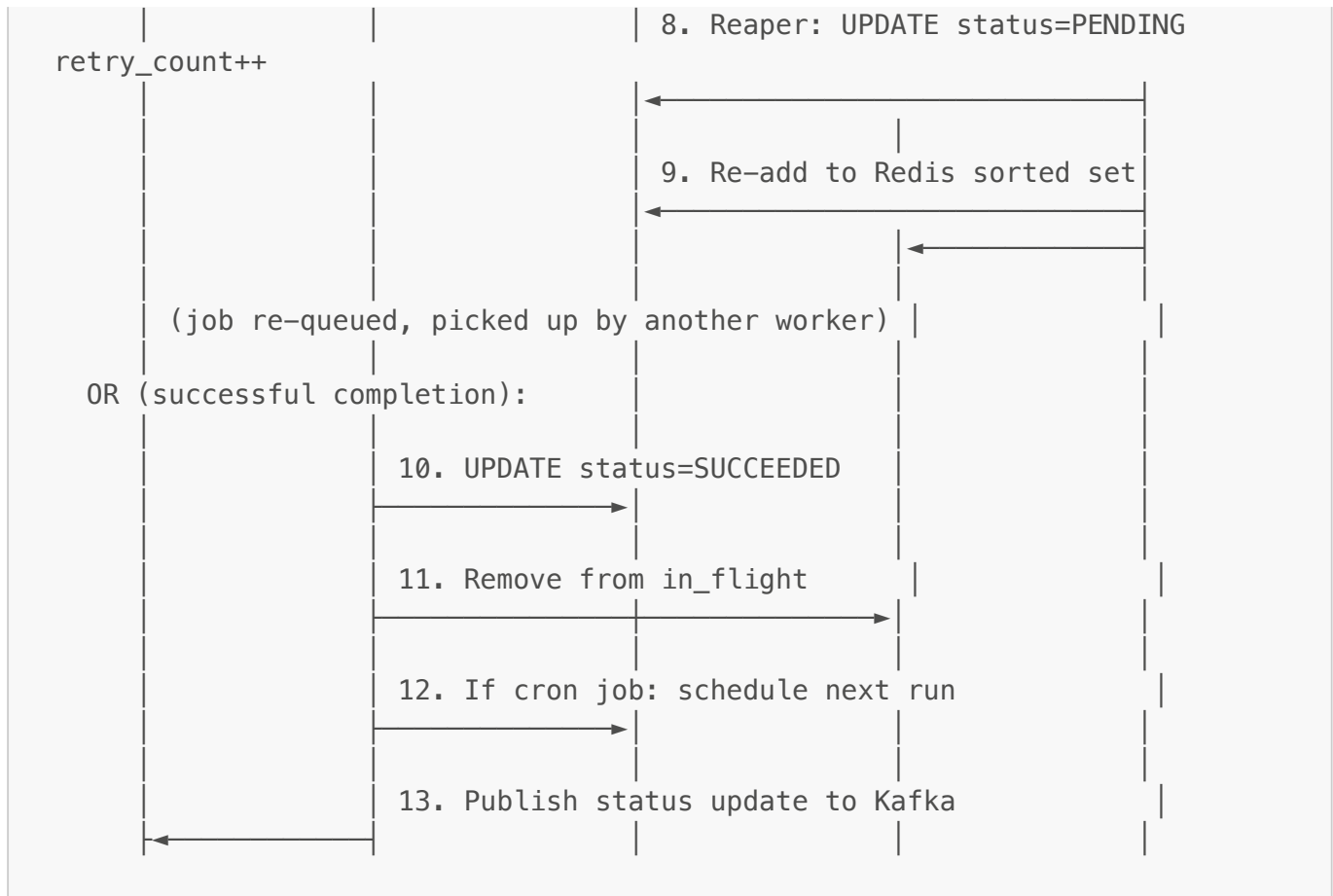
- Client submits: `cron='0 9 * * 1-5', timezone='America/New_York'`
- We convert each calculated `next_run_at` to UTC before storing

This means a job that says 'run at 9 AM New York time' continues to run at 9 AM New York time even during daylight saving time transitions, because we recalculate the UTC time before each occurrence."

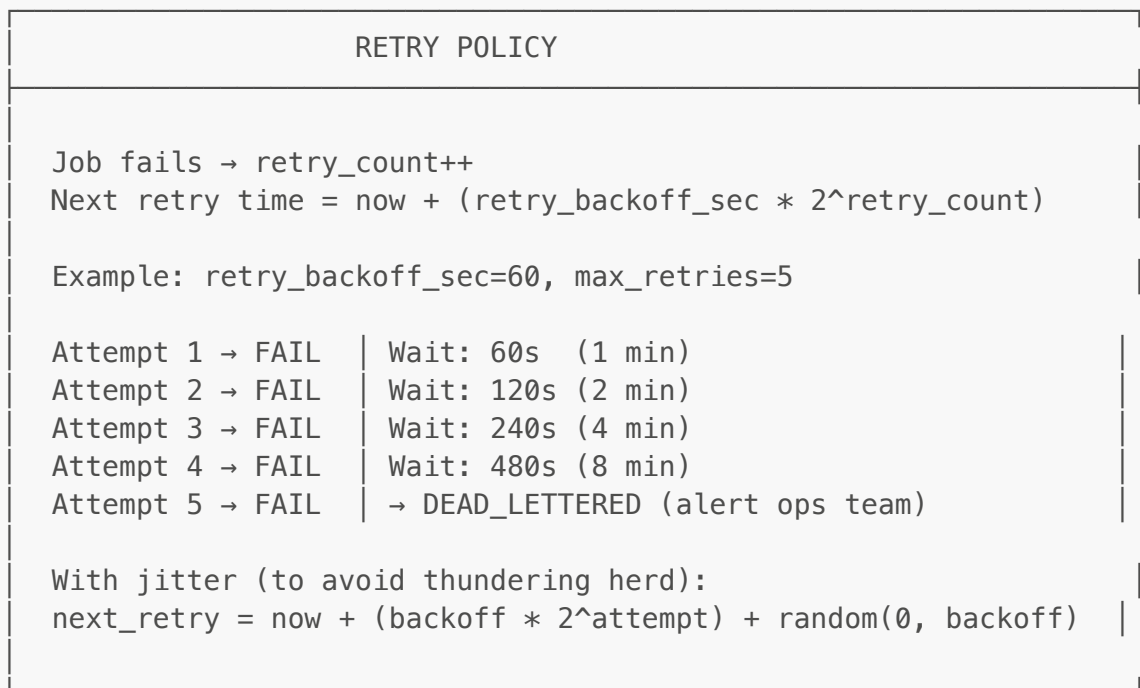
SECTION 5: JOB EXECUTION FLOW

5.1 Worker Execution Sequence Diagram





5.2 Retry Logic with Exponential Backoff



```
// Java retry calculation
public Instant calculateNextRetry(Job job) {
    long baseDelay = job.getRetryBackoffSec();
```

```

    int attempt = job.getRetryCount();
    long jitter = ThreadLocalRandom.current().nextLong(0, baseDelay);
    long delaySeconds = (long)(baseDelay * Math.pow(2, attempt)) + jitter;
    // Cap at 1 hour max
    delaySeconds = Math.min(delaySeconds, 3600);
    return Instant.now().plusSeconds(delaySeconds);
}

```

CROSS-QUESTIONS & ANSWERS:

Q1: How do you handle jobs that need to run on a specific server (data locality)? "This is called **worker affinity** or **task routing**. Use cases:

- A video transcoding job needs a worker with a GPU
- A file processing job needs a worker with specific mounted storage

Solution: Tagged Workers + Job Routing

```

Job payload:  { 'requires': ['gpu', 'high-memory'] }
Worker tags:  worker-A: ['gpu', 'high-memory']
               worker-B: ['standard']
               worker-C: ['gpu']

Kafka topic per capability:
  jobs.capability.gpu
  jobs.capability.standard

Workers subscribe only to topics matching their capabilities.

```

The scheduler routes GPU jobs to **jobs.capability.gpu** topic. Only workers with GPU tags consume from that topic."

Q2: How do you handle long-running jobs (like an ML training job that takes 6 hours)? "Standard timeout-and-retry would keep retrying a 6-hour job incorrectly. Solutions:

- 1. Extended Heartbeat Timeout:** Per-job configurable timeout: **timeout_s=21600** (6 hours). Reaper only considers the job stale if no heartbeat for > timeout_s.
- 2. Progress Checkpointing:** Long jobs periodically write checkpoints into the payload JSONB field. On restart, the worker reads the checkpoint and resumes from the last saved position instead of starting over.
- 3. Job Fragmentation:** Break a 6-hour job into 100 sub-jobs of 3.6 minutes each, with job dependencies. If one sub-job fails, only that fragment is retried."

SECTION 6: HANDLING SCALE & RELIABILITY

6.1 Partitioning Jobs for Scale

HORIZONTAL SCALING STRATEGY

Problem: 1M jobs due at midnight (batch day-end processing)

Solution: TIME BUCKET SHARDING

Divide the timeline into 1-minute buckets:

Bucket 0: 00:00:00 – 00:00:59 → Scheduler Node 0 owns this

Bucket 1: 00:01:00 – 00:01:59 → Scheduler Node 1 owns this

Bucket 2: 00:02:00 – 00:02:59 → Scheduler Node 2 owns this

...

Each Redis sorted set is partitioned:

due_jobs:00 → jobs due in minute 0

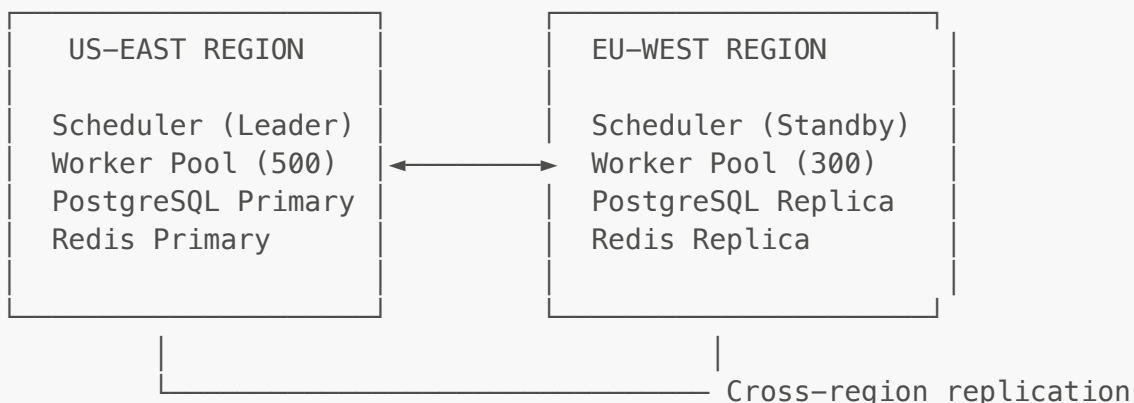
due_jobs:01 → jobs due in minute 1

Scheduler node 0 only polls due_jobs:00

→ 1M jobs split across 10 nodes = 100K jobs each = manageable

6.2 Multi-Region Deployment

MULTI-REGION SETUP



RULES:

- Jobs are created in the closest region (low latency for submission)
- Job execution is regional by default (data sovereignty)
- Cross-region fallback: if US-East goes down, EU-West scheduler takes over US-East jobs within 30 seconds (Redlock times out)
- PostgreSQL: active-passive with streaming replication, RPO < 1 second

6.3 Rate Limiting & Throttling

RATE LIMITING STRATEGIES

1. CLIENT-LEVEL RATE LIMITING (API Gateway)
Max 100 job submissions / second / client
Redis token bucket: 100 tokens/sec, burst=500
2. JOB TYPE THROTTLING
Max concurrent jobs per type:
 email_send: 500 concurrent
 report_gen: 50 concurrent
 data_export: 20 concurrent
Prevents one type from starving others
3. WORKER POOL AUTOSCALING
Scale out: Kafka consumer lag > 1000 messages
Scale in: Kafka consumer lag < 100 messages for 5 mins
4. BACKPRESSURE
If Kafka queue depth > 500K messages:
 - Reject new low-priority job submissions
 - Return HTTP 429 with retry-after header
 - High and medium priority still accepted

CROSS-QUESTIONS & ANSWERS:

Q1: How does your system compare to existing tools like Quartz, Celery, AWS SQS+Lambda?

Tool	Strengths	Weaknesses
Quartz (Java)	Mature, Java-native, clustered Cron support	JVM only, no dashboard Complex clustering config
Celery (Python)	Multi-language, distributed Redis/RabbitMQ backend	Python-centric, complex monitoring setup
AWS SQS + Lambda	Managed, highly available No infra to manage	No cron natively, Cold start latency
Airflow	Complex DAGs, UI, data lineage, monitoring	Heavyweight, not for simple job scheduling
Our Design	Custom control, priority queues, job dependencies	Need to build and maintain it yourself

"Our custom design makes sense when:

1. You need features that off-the-shelf tools don't support (custom priority tiers, complex retry policies, multi-tenant job isolation)
2. You need tight integration with your existing stack
3. At scale where managed services become prohibitively expensive

For most companies starting out, I'd actually recommend Quartz + PostgreSQL or AWS EventBridge + SQS + Lambda. Build custom when you've outgrown those."

Q2: How do you prevent a 'hot shard' when many jobs have the same target time? "Classic thundering herd problem. Three strategies:

1. Scheduled-Time Jitter: When a user says 'run every hour', instead of running at exactly :00:00, we add random jitter:

```
scheduled_at = requested_time + random(-30, +30) seconds
```

10,000 jobs scheduled for midnight end up spread across 11:59:30 PM to 12:00:30 AM.

2. Priority-Ordered Execution: Even if 100,000 jobs arrive simultaneously, workers process them in priority order. The queue absorbs the burst.

3. Smooth Draining: Kafka partitions provide natural load distribution. With 10 partitions and 100 workers, each worker gets 1/10th of the simultaneous burst."

SECTION 7: API DESIGN

7.1 REST API Endpoints

BASE URL: /api/v1

JOB MANAGEMENT:

POST	/jobs	→ Submit new job
GET	/jobs/{id}	→ Get job status and metadata
PUT	/jobs/{id}	→ Update job (if still PENDING)
DELETE	/jobs/{id}	→ Cancel job
GET	/jobs	→ List jobs (paginated, filterable)
POST	/jobs/{id}/pause	→ Pause a recurring job
POST	/jobs/{id}/resume	→ Resume a paused job
POST	/jobs/{id}/trigger	→ Force-trigger a job now

EXECUTION HISTORY:

GET	/jobs/{id}/executions	→ List all execution attempts
GET	/jobs/{id}/executions/{exec_id}	→ Get specific execution detail

WORKER MANAGEMENT (Admin):

```
GET    /admin/workers      → List all workers and status
GET    /admin/workers/{id} → Get worker detail
POST   /admin/workers/{id}/drain → Gracefully stop a worker
```

METRICS:

```
GET    /metrics/queue-depth → Current queue depths by priority
GET    /metrics/failure-rate → Job failure rates by type
```

7.2 Request / Response Examples

```
// POST /api/v1/jobs - Submit one-time job
```

Request:

```
{
  "name": "send-monthly-invoice",
  "job_type": "email",
  "priority": 3,
  "scheduled_at": "2025-08-01T00:00:00Z",
  "payload": {
    "user_id": "usr_123",
    "template": "invoice_monthly",
    "invoice_id": "inv_456"
  },
  "max_retries": 3,
  "retry_backoff_sec": 60,
  "timeout_sec": 30
}
```

Response (201 Created):

```
{
  "id": "job_9f3c2a1b",
  "status": "PENDING",
  "scheduled_at": "2025-08-01T00:00:00Z",
  "created_at": "2025-07-15T10:23:44Z",
  "estimated_queue_position": 1234
}
```

```
// POST /api/v1/jobs - Submit recurring job (cron)
```

Request:

```
{
  "name": "nightly-db-backup",
  "job_type": "database_backup",
  "priority": 2,
  "cron_expression": "0 2 * * *",
  "timezone": "UTC",
  "payload": {
    "db_name": "production",
    "s3_bucket": "backups-prod",
    "retention_days": 30
  },
}
```

```
"max_retries": 5
}

// GET /api/v1/jobs/job_9f3c2a1b
Response:
{
  "id": "job_9f3c2a1b",
  "name": "send-monthly-invoice",
  "status": "SUCCEEDED",
  "priority": 3,
  "scheduled_at": "2025-08-01T00:00:00Z",
  "started_at": "2025-08-01T00:00:00.512Z",
  "finished_at": "2025-08-01T00:00:01.234Z",
  "duration_ms": 722,
  "retry_count": 0,
  "worker_id": "wrk_a1b2c3d4",
  "executions": [
    {
      "attempt": 1,
      "status": "SUCCEEDED",
      "started_at": "2025-08-01T00:00:00.512Z",
      "exit_code": 0
    }
  ]
}
```

SECTION 8: MONITORING & OBSERVABILITY

8.1 Key Metrics to Track

GOLDEN SIGNALS FOR JOB SCHEDULER

LATENCY (Scheduling Accuracy)

`scheduler_trigger_delay_ms` → `actual_start - scheduled_at`

Target: p99 < 1000ms, p50 < 200ms

Alert: p99 > 5000ms

ERRORS

`job_failure_rate_by_type` → `failures / total per job_type`

Target: < 0.1%

Alert: > 1% for any type

`dead_letter_rate` → jobs sent to dead letter queue

Alert: > 0 dead-lettered jobs in 5 minutes

SATURATION

```

kafka_consumer_lag      → messages behind in queue
Target: < 1000 messages
Alert: > 10000 (scale out workers)

worker_utilization      → busy workers / total workers
Alert: > 90% for 5 min (auto-scale trigger)

TRAFFIC

jobs_submitted_per_sec  → submission rate
jobs_completed_per_sec  → throughput
jobs_scheduled_per_sec  → scheduler dispatch rate

```

8.2 Grafana Dashboard Layout

JOB SCHEDULER DASHBOARD											
Queue Depth High: 1,234 Med: 4,567 Low: 12,890	Trigger Delay (p99) 187ms  OK	Active Workers 892 / 1000 89% utilization → scaling up									
Jobs/sec (24hr)  Peak: 1,100/sec	Failure Rate 0.08%  Good	Dead Letter Queue 0 jobs ✓ All good									
Top Failed Job Types (last 1hr)											
<table> <tr> <td>email_send:</td><td>23 failures</td><td>(timeout: recipient server slow)</td></tr> <tr> <td>report_gen:</td><td>3 failures</td><td>(OOM: report too large)</td></tr> <tr> <td>data_export:</td><td>1 failure</td><td>(permission denied on S3 path)</td></tr> </table>			email_send:	23 failures	(timeout: recipient server slow)	report_gen:	3 failures	(OOM: report too large)	data_export:	1 failure	(permission denied on S3 path)
email_send:	23 failures	(timeout: recipient server slow)									
report_gen:	3 failures	(OOM: report too large)									
data_export:	1 failure	(permission denied on S3 path)									

CROSS-QUESTIONS & ANSWERS:

Q1: How do you debug a job that's stuck in RUNNING state for 2 hours when it should finish in 5 minutes? "This is a common production incident. Debugging steps:

Step 1: Check worker heartbeat Query the workers table joining to the stuck job — check last_heartbeat. If the worker is alive but the job is slow, it's a performance issue. If no heartbeat in >30s, the reaper should have recovered it already.

Step 2: Check execution logs Query job_executions for the latest attempt's output and error_message columns.

Step 3: SSH into the worker Check CPU/memory. Is the process still running? Is it blocked on a network call?

Step 4: Force-timeout the job Update the job status to FAILED with an error_message explaining the manual override. This triggers retry logic normally.

Root cause prevention: Every external call inside a job must have an explicit timeout. Never rely on the default which is often 'wait forever'."

SECTION 9: FAILURE SCENARIOS & EDGE CASES

9.1 Failure Mode Analysis

FAILURE MODE ANALYSIS		
Failure	Impact	Mitigation
Scheduler dies	No new jobs dispatched for up to 5 seconds	Redlock leader elect Standby takes over
Worker crashes	In-progress job lost temporarily	Heartbeat + reaper retries within 30s
Redis goes down	Can't dispatch new jobs lose in-memory schedule	Fallback to PostgreSQL polling (slower)
Kafka goes down	Jobs dispatched but not consumed	Kafka acks before marking QUEUED
PostgreSQL down	Can't create/update jobs	Redis buffers state, sync when DB recovers
Clock skew between nodes	Jobs trigger at wrong times	NTP sync, use UTC, tolerate 100ms skew
Duplicate job dispatch	Same job runs twice	Idempotency key in payload + DB UPDATE WHERE status=QUEUED

9.2 Redis Fallback Mode

NORMAL MODE:

Client → API → Redis Sorted Set → Scheduler polls Redis → Kafka → Worker

REDIS DOWN MODE (graceful degradation):

Client → API → PostgreSQL only → Scheduler polls PostgreSQL → Kafka → Worker

```
PostgreSQL fallback query:
SELECT id FROM jobs
WHERE status = 'PENDING'
      AND scheduled_at <= NOW()
ORDER BY priority ASC, scheduled_at ASC
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

Trade-off: higher DB load, slightly slower (50ms vs 1ms), but no jobs lost. Scheduler detects Redis is down via connection health check and flips to fallback mode.

CROSS-QUESTIONS & ANSWERS:

Q1: How do you guarantee a job executes exactly once? Isn't at-least-once risky? "You're right that at-least-once can cause issues. Here's the full picture:

At-least-once is a scheduler guarantee, not an application guarantee.

The application (the job itself) must be idempotent. Examples:

```
// NON-IDEMPOTENT (dangerous):
void sendWelcomeEmail(String userId) {
    emailService.send(userId, "Welcome!"); // sends twice if retried!
}

// IDEMPOTENT (safe):
void sendWelcomeEmail(String userId, String jobId) {
    if (!emailLog.exists(jobId)) {           // check if already sent
        emailService.send(userId, "Welcome!");
        emailLog.save(jobId);                // record that we sent it
    }
}
```

The jobId is passed as the idempotency key. Even if the job runs twice, the second run is a no-op.

For financial operations, use an idempotency_key column with a unique constraint — the second INSERT is silently ignored by the database."

Q2: What if a job's schedule is every minute but it takes 90 seconds to run? "This is the **job overlap** problem. Three strategies:

Option 1: Skip (Default) If the previous instance is still running when the next trigger fires, skip this occurrence. Use a distributed lock:

```
SET job:lock:{job_id} {execution_id} NX PX 60000
```

Second trigger sees lock exists → skips this run.

Option 2: Queue (Allow Overlap) Let both instances run concurrently. Fine for idempotent jobs.

Option 3: Replace (Cancel Previous) Kill the running instance, start fresh. Used for jobs where you always want the latest data.

Best Practice: Default to 'skip' with an alert when skip occurs, so you know the job is taking too long and needs optimization."

SECTION 10: INTERVIEW CHEATSHEET

10.1 Quick-Reference Summary

JOB SCHEDULER – INTERVIEW QUICK REFERENCE	
CORE COMPONENTS	
API Service	→ Job CRUD, submission, status query
Scheduler	→ Polls Redis sorted set every 1 sec, dispatches
Kafka	→ Priority queues, decouples dispatch from exec
Workers	→ Execute jobs, heartbeat every 5s
Reaper	→ Detects dead workers, re-queues orphaned jobs
PostgreSQL	→ Source of truth, audit log
Redis	→ Fast scheduling index, leader election, locks
KEY DESIGN CHOICES	
At-least-once	→ Retry on failure, require job idempotency
Redis + PG	→ Speed of Redis + durability of Postgres
Redlock	→ Single leader prevents double dispatch
Priority Kafka	→ High-priority jobs never starved by low-priority
Exponential backoff	→ Backoff with jitter prevents thundering herd
COMMON TRICKS / GOTCHAS	
1. Jobs at same time → jitter to spread load	
2. Worker crash → heartbeat timeout + reaper	
3. Double dispatch → atomic Lua script in Redis	
4. Cron overlap → distributed lock, default to skip	
5. Time zones → store UTC, convert at scheduling time	
6. Redis down → fall back to PostgreSQL polling	
NUMBERS TO REMEMBER	
10M jobs/day = 116/sec avg, 1,000/sec peak	
Heartbeat: every 5 sec	
Stale worker timeout: 30 sec	

```
Leader lock TTL: 5 sec (renewed every 2 sec)
Target trigger delay: p99 < 1 second
Max concurrent workers: 1,000
```

10.2 Top 10 Interview Questions with Answers

Q1: "Design a job scheduler. Where do you start?" "I always start with clarifying requirements. Key questions:

- One-time or recurring jobs? (Both)
- How many jobs per second? (Sets scale needs)
- What's the acceptable delay after scheduled time? (Defines accuracy SLA)
- What happens if a job fails? (Retry policy)
- Do jobs have dependencies on each other? (Changes design significantly)

Then I'd define non-functional requirements, estimate capacity, and draw the high-level architecture."

Q2: "How do you prevent the same job from running twice?" "Three layers of defense:

1. **Scheduler layer:** Atomic Lua script in Redis removes job from due_jobs before dispatching
2. **Database layer:** UPDATE jobs SET status='RUNNING' WHERE status='QUEUED' - if 0 rows affected, another worker already claimed it
3. **Application layer:** Idempotency key in job payload, checked before executing logic"

Q3: "What happens to a job if the worker executing it crashes?" "The heartbeat + reaper pattern handles this:

- Worker sends heartbeat to Redis every 5 seconds
- Reaper process checks for workers with stale heartbeats (>30s old)
- Reaper marks the worker as DEAD and the job as PENDING
- Job is re-added to Redis sorted set for re-execution
- Recovery time: up to 30 seconds"

Q4: "How do you handle 1 million jobs all scheduled for the same time?" "Multiple strategies:

1. **Add jitter** at scheduling time: ± 30 seconds random offset
2. **Priority queues absorb burst:** Kafka queues handle millions of messages
3. **Auto-scale workers:** Monitor Kafka consumer lag, spin up workers proactively
4. **Time bucket sharding:** Multiple schedulers each own a time window"

Q5: "Why Redis sorted set and not a regular priority queue?" "A Redis sorted set is essentially a priority queue where the 'priority' is the scheduled time (epoch ms). It gives us:

- $O(\log N)$ insertion (ZADD)
- $O(\log N)$ range query for due jobs (ZRANGEBYSCORE 0 now)

- Natural expiration of old entries
- Shared across multiple scheduler nodes Standard in-memory priority queues don't work across distributed nodes."

Q6: "How do you handle a recurring job that takes longer than its interval?" "Default strategy is 'skip with alert'. Use a distributed lock per job_id:

- When an instance starts: acquire lock `SET job:running:{id} 1 NX PX {interval_ms}`
- If lock already exists, this occurrence is skipped
- Alert ops team when skip rate exceeds threshold (job needs optimization)"

Q7: "How would you add support for job dependencies (Job B only after Job A)?" "Add a job_dependencies table with (job_id, depends_on_job_id). The scheduler checks if all dependency jobs have status=SUCCEEDED before dispatching. If not ready, the scheduler skips this job and checks again on the next poll cycle. This is how Airflow DAGs work at a basic level."

Q8: "How do you ensure high availability of the scheduler itself?" "Run multiple scheduler nodes. Use Redlock to elect a single leader:

- All nodes try: `SET scheduler:leader {node_id} NX PX 5000`
- Only one wins
- Winner dispatches jobs, renews lock every 2 seconds
- If leader dies, lock expires in 5 seconds
- A standby node wins the next election Max downtime: ~5 seconds, within our 99.99% SLA."

Q9: "How do you monitor if the scheduler is healthy?" "Three key signals:

1. **Trigger delay:** `actual_start_time - scheduled_at`. Alert if p99 > 5 seconds.
2. **Kafka consumer lag:** If lag grows, workers aren't keeping up. Trigger auto-scaling.
3. **Dead letter queue:** Any job landing here is a bug/alert. DLQ size should always be 0."

Q10: "Why not just use AWS Lambda + CloudWatch Events/EventBridge?" "Valid option for many cases. Use managed services when:

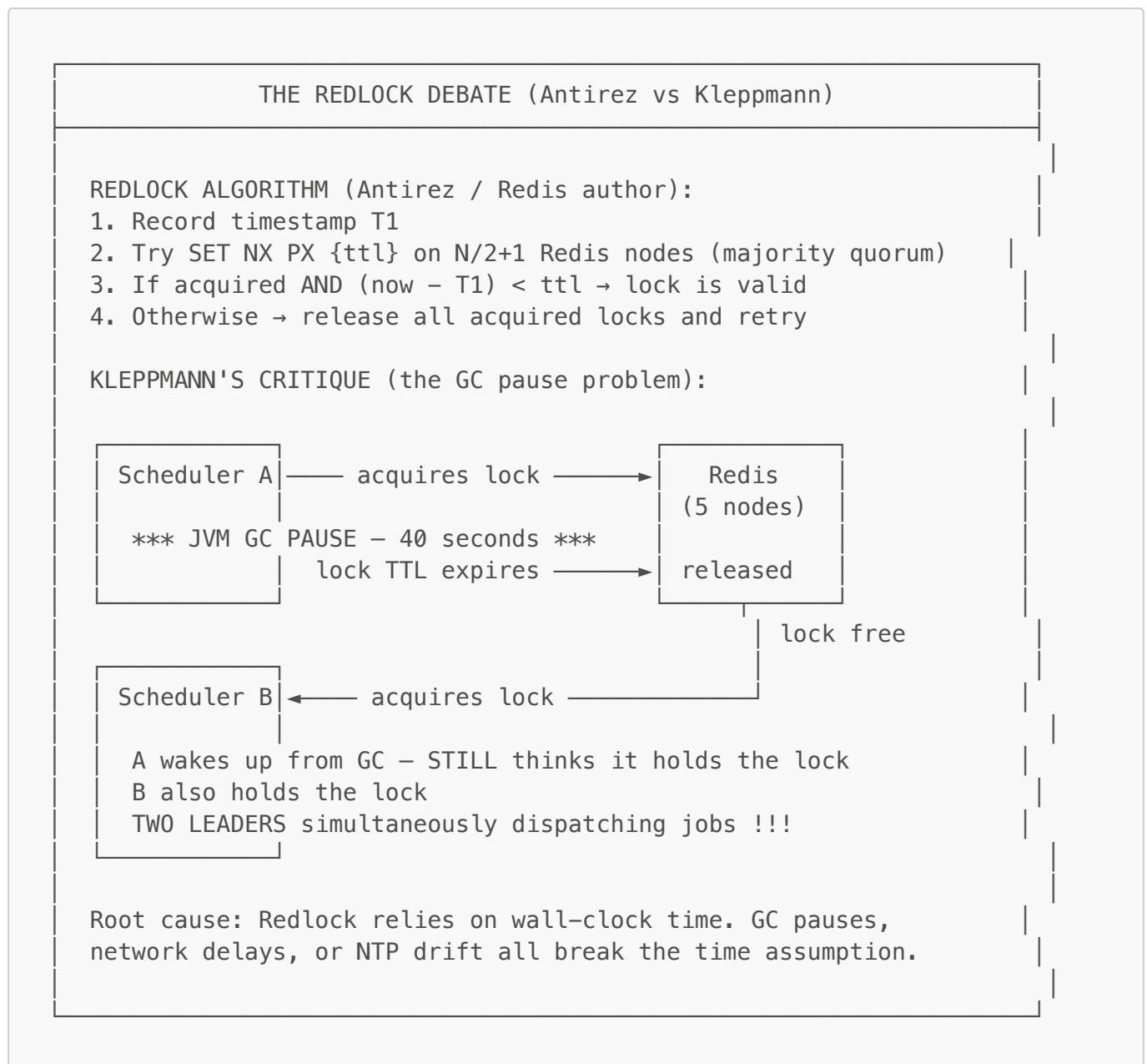
- Scale is moderate (<1M jobs/day)
- No need for priorities or complex retry policies
- Fine with Lambda cold starts (~100-500ms latency)
- Team wants less infrastructure to manage

Build custom when:

- Need sub-second scheduling accuracy
- Need complex features: priorities, dependencies, multi-tenant isolation
- Cost at massive scale: 10M Lambda invocations/day has significant cost
- Need full control over execution environment (GPUs, memory, CPU)"

SECTION 11: PRINCIPAL-LEVEL DEEP DIVES

11.1 Redlock Controversy — Is It Actually Safe?



How to Explain: "This is one of my favorite topics because there's a famous public debate between Antirez — the creator of Redis — and Martin Kleppmann, who wrote *Designing Data-Intensive Applications*.

Antirez designed Redlock as a distributed lock across 5 Redis nodes. The idea: if you can acquire the lock on 3 out of 5 nodes within the TTL, you hold the lock. That sounds solid.

Kleppmann's critique is subtle but devastating. The problem isn't Redis — it's time. Imagine Scheduler A acquires the Redlock, then the JVM has a garbage collection pause for 40 seconds. During that pause, the lock TTL expires. Redis releases it. Scheduler B grabs the lock. Now A wakes up from GC — it has no idea time passed. It thinks it still holds the lock. Two schedulers are now both dispatching jobs simultaneously.

So is Redlock broken for our use case? Honestly — it's good enough. Here's why: even if two schedulers both dispatch the same job, our DB optimistic lock saves us:

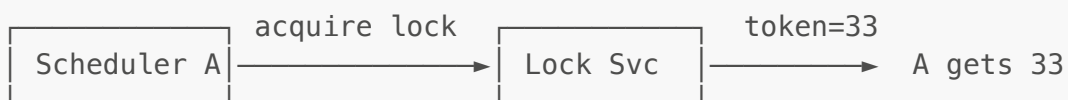
```
UPDATE jobs SET status='QUEUED' WHERE id=? AND status='PENDING'
```

Only one scheduler sees rowcount=1. The other gets 0 and skips it. So the DB is our real safety net.

But let me tell you the correct solution for systems that need absolute guarantees — like a payment processor:"

FENCING TOKENS – The Correct Fix

Lock server returns a monotonically increasing token on each grant:



A writes to DB: `UPDATE jobs SET ... WHERE id=? AND lock_token < 33`
DB stores token=33 on the row

*** A gets GC paused, lock expires ***



B writes to DB: `UPDATE ... WHERE lock_token < 34` ✓ succeeds
DB stores token=34 on the row

A wakes up, tries: `UPDATE ... WHERE lock_token < 33`
DB rejects! `33 < 34` is false – stale write blocked ✓

Even if A thinks it holds the lock, the DB rejects stale writes.
Split-brain damage is impossible, regardless of clock drift.

"The fencing token approach eliminates the time assumption entirely. The DB enforces ordering, not the clock. For our job scheduler, Redlock plus DB optimistic locking is fine. For a financial ledger, I would use fencing tokens — or skip Redis entirely and use etcd or ZooKeeper, which are built on Raft consensus and give you actual CP guarantees."

11.2 Table Partitioning & Archival at Scale

How to Explain: "Let me walk through something most people miss. Our jobs table grows at 10 million rows per month. After 12 months that's 120 million rows. After 3 years — 360 million. At some point the naive cleanup approach completely breaks down.

The naive approach is:

```
DELETE FROM jobs WHERE status='SUCCEEDED' AND created_at < NOW() - INTERVAL '90 days'
```

At 120 million rows, this DELETE takes minutes, holds locks, spikes I/O, and creates replication lag on standby replicas. I've seen this take production databases down.

The correct approach is table partitioning:"

RANGE PARTITIONING BY scheduled_at (PostgreSQL declarative)

jobs (parent)

- └ jobs_2025_06 → all jobs with scheduled_at in June 2025
- └ jobs_2025_07 → July 2025
- └ jobs_2025_08 → August 2025 ← hot partition, all writes here
- └ jobs_2025_09 → pre-created before Aug 25th

SCHEDULER POLL QUERY:

```
SELECT * FROM jobs
WHERE scheduled_at <= NOW() AND status='PENDING'
```

PostgreSQL partition pruning:

- Skips jobs_2025_06, jobs_2025_07 entirely
- Only scans jobs_2025_08 (today's partition)
- 100K rows scanned instead of 120M ← massive speedup

ARCHIVAL (quarterly maintenance job):

- Step 1: pg_dump partition → upload to S3 as Parquet
- Step 2: DROP TABLE jobs_2025_06 ← INSTANT, no row iteration
- Step 3: History queryable on S3 via Athena for compliance

MAINTENANCE JOB (runs monthly on the 25th):

- ```
CREATE TABLE jobs_NEXT_MONTH PARTITION OF jobs
FOR VALUES FROM ('2025-09-01') TO ('2025-10-01');
```
- Never caught without a partition

## CROSS-QUESTIONS & ANSWERS:

**Q: How do you handle a job submitted in August that's scheduled to run in September?** "Great catch. The partition is determined by `scheduled_at`, not `created_at`. So a job submitted today but



scheduled for September 15th gets inserted directly into `jobs_2025_09` at submission time. This is why pre-creating next month's partition before month-end is critical — if that partition doesn't exist, the INSERT fails. I'd run the partition creation job on the 25th of each month as a safety margin."

### 11.3 Poison Pill Detection & Job Quarantine

**How to Explain:** "Here's an edge case that will take down your worker pool if you don't plan for it. Imagine a job has a memory leak that causes an OOM kill. Worker picks it up, crashes. Reaper detects the dead worker, re-queues the job. Another worker picks it up, crashes. And again. You're stuck in an infinite crash loop, and each iteration kills a healthy worker.

This is called a poison pill. Let me show you how I'd detect and quarantine it:"

#### POISON PILL DETECTION

SYMPTOM (what you see in logs):

```
job_xyz → Worker-1 claims it → Worker-1 DEAD (OOM)
job_xyz → Worker-2 claims it → Worker-2 DEAD (OOM)
job_xyz → Worker-3 claims it → Worker-3 DEAD (OOM)
Reaper keeps re-queuing... destroying the pool
```

CIRCUIT BREAKER IN REAPER:

When Reaper detects a dead worker for job\_xyz:

```
INCR job:worker_deaths:{job_id} → atomically increment
EXPIRE job:worker_deaths:{job_id} 86400 → reset daily
```

```
deaths = GET job:worker_deaths:{job_id}
```

IF deaths >= 3:

```
UPDATE jobs SET status='QUARANTINED' WHERE id='{job_id}'
Alert ops: "☠ Poison pill detected: job_xyz killed 3 workers"
Do NOT re-queue
```

IF deaths < 3:

```
Normal exponential backoff retry
```

QUARANTINED JOB LIFECYCLE:

```
Ops reviews: worker crash logs + job payload
Fix deployed → POST /admin/jobs/{id}/release
Counter reset → job retried normally
Or: auto-release when new code version deployed
```

"The beauty of using Redis INCR here is that it's atomic. Even if 3 workers die simultaneously and 3 Reaper threads run concurrently, INCR guarantees exactly one thread sees the count reach 3 and triggers the quarantine. No race condition."

## 11.4 Graceful Worker Drain (Zero-Downtime Deploys)

**How to Explain:** "Here's a scenario that causes data loss in production if you haven't thought about it. You push a new deployment. Kubernetes sends SIGTERM to each worker pod to replace it. If the worker is in the middle of running a 10-minute job when SIGTERM arrives, and you just kill the process — what happens? The job is in RUNNING state with no worker finishing it. The Reaper detects a dead worker in 30 seconds and re-queues it. The job runs twice.

The solution is graceful drain — let me draw it:"

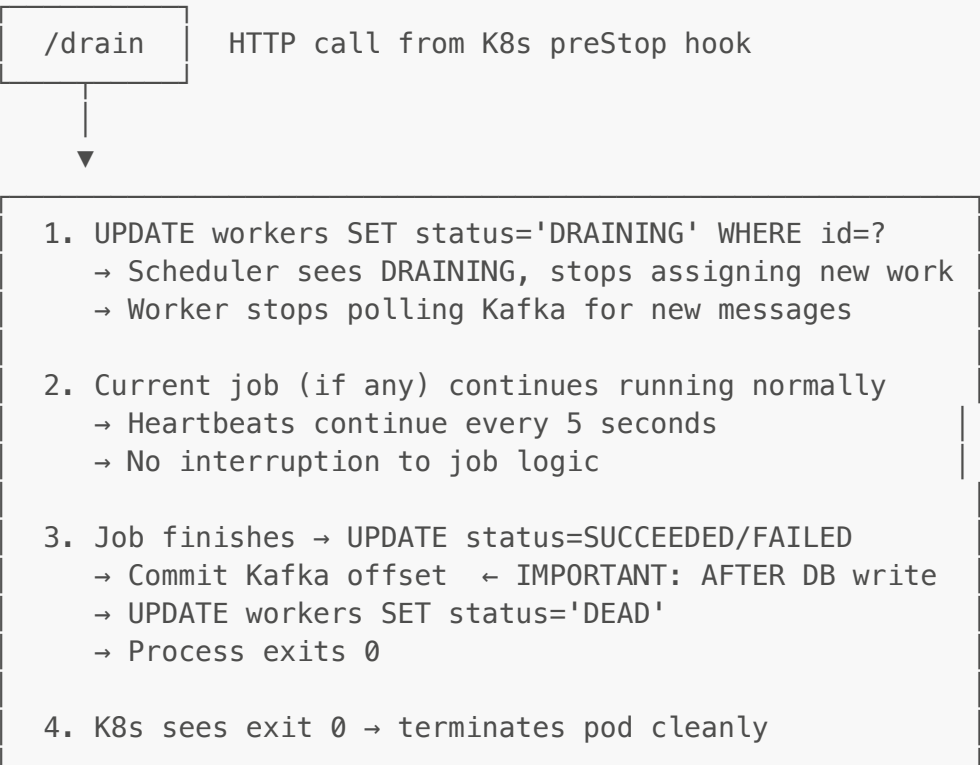
### GRACEFUL WORKER DRAIN — Zero-Downtime Rolling Deploy

Kubernetes manifest:

```
terminationGracePeriodSeconds: 3600 # wait up to 1 hr
lifecycle:
 preStop:
 httpGet: { path: /worker/drain, port: 8082 }
→ K8s calls /drain BEFORE sending SIGTERM
```

DRAIN SEQUENCE:

`/drain` HTTP call from K8s preStop hook

- 
1. UPDATE workers SET status='DRAINING' WHERE id=?  
→ Scheduler sees DRAINING, stops assigning new work  
→ Worker stops polling Kafka for new messages
  2. Current job (if any) continues running normally  
→ Heartbeats continue every 5 seconds  
→ No interruption to job logic
  3. Job finishes → UPDATE status=SUCCEEDED/FAILED  
→ Commit Kafka offset ← IMPORTANT: AFTER DB write  
→ UPDATE workers SET status='DEAD'  
→ Process exits 0
  4. K8s sees exit 0 → terminates pod cleanly

IF WORKER IS IDLE (no current job):

```
/drain called → no job running → exit immediately
terminationGracePeriod not consumed at all
```

"There's a subtle ordering issue with Kafka that bites people. You must commit the Kafka offset AFTER you've successfully updated the job status to SUCCEEDED in the database — not before. If you commit offset first and then the process dies before the DB update, the next worker never picks up the job because Kafka thinks it was processed. That's an at-most-once bug in an at-least-once system. Commit order: DB first, Kafka offset second."

## 11.5 Fan-Out Jobs & Parent Completion Tracking

**How to Explain:** "Let's talk about fan-out. Real-world example: 'Send the monthly invoice email to all 10 million users.' That's one parent job that spawns 10 million child email jobs. The product team wants a Slack notification when all 10 million emails are sent. How do you track completion?"

The naive approach is what most people reach for first:"

NAIVE APPROACH (kills your database):

Every 5 seconds, poll:

```
SELECT COUNT(*) FROM jobs
WHERE parent_id = 'invoice_job_aug'
AND status != 'SUCCEEDED'
```

Problem: Full index scan over 10M rows, every 5 seconds.  
At 10M concurrent workers updating rows, this query fights with millions of UPDATE locks. Database collapses.

CORRECT APPROACH: Atomic distributed counter in Redis

Fan-out creation (parent job spawns children):

```
SET job:fanout:pending:{parent_id} 10000000 EX 86400
SET job:fanout:failed:{parent_id} 0 EX 86400
```

Each child job on SUCCESS:

```
remaining = DECR job:fanout:pending:{parent_id}
IF remaining == 0:
 publish to Kafka: { type: "fanout_complete",
 parent_id: ...,
 failed_count: GET job:fanout:failed }
```

Each child job DEAD\_LETTERED (exhausted retries):

```
INCR job:fanout:failed:{parent_id}
remaining = DECR job:fanout:pending:{parent_id}
```

```
IF remaining == 0: same completion event with failures
```

Why this works:

Redis DECR is atomic → 10M concurrent DECRs are safe

Exactly one thread sees remaining==0 → fires completion event

No polling, no DB lock contention

"For more complex DAG dependencies — where Job C depends on both Job A and B — the `job_dependencies` table handles it. The scheduler checks: are all my dependencies SUCCEEDED? If yes, dispatch. The DECR counter pattern still applies per level of the DAG. Redis does the counting, the DB holds the graph structure."

## 11.6 Multi-Tenant Noisy Neighbor

**How to Explain:** "Imagine we're selling this job scheduler as a SaaS platform. Tenant A is a large enterprise. On the last day of the month, they submit 900,000 reporting jobs at midnight. Our Kafka queues fill up. Tenant B is a small startup with 100,000 jobs. Their jobs sit behind Tenant A's 900K for hours. Tenant B is paying the same SLA. This is the noisy neighbor problem."

Let me walk through three solutions from simplest to most sophisticated:"

### SOLUTION 1: Per-Tenant Kafka Topics

`jobs.high_priority.{tenant_id}` – dedicated topic per tenant

Worker pools assigned per tenant tier

Problem: 1,000 tenants = 1,000+ topics. Kafka management hell.

Verdict: Only works for small tenant counts (<20 large tenants)

### SOLUTION 2: Per-Tenant Concurrency Quota ← I'd recommend this

`tenant_quotas` table:

| tenant_id | max_concurrent | plan       |
|-----------|----------------|------------|
| tenant_a  | 500            | enterprise |
| tenant_b  | 200            | startup    |
| tenant_c  | 50             | free       |

Redis quota enforcement (at worker claim time):

```
running = GET tenant:running:{job.tenant_id}
```

```
limit = tenant_quotas[job.tenant_id].max_concurrent
```

```
IF running >= limit:
```

```

 skip this job → try next job from a different tenant
ELSE:
 INCR tenant:running:{tenant_id}
 claim the job

```

On job complete: DECR tenant:running:{tenant\_id}

Result: Tenant A uses all 500 slots but cannot use slot 501.  
Tenant B always has their 200 slots available.

SOLUTION 3: Weighted Fair Queue (for strict fairness)

$\text{effective\_priority} = \text{base\_priority} + (\text{wait\_seconds} * \text{weight})$

Tenant A job waiting 1 second:  $\text{priority} = 5 + (1 \times 1.0) = 6.0$

Tenant B job waiting 60 seconds:  $\text{priority} = 5 + (60 \times 1.0) = 65$

Even with no explicit quota, the longer a job waits,  
the higher its priority climbs → starvation is mathematically  
impossible regardless of submission rate

"In practice, I'd combine solutions 2 and 3: hard quotas to prevent noisy neighbors, plus weighted fair queuing within each tenant's quota to prevent internal starvation."

## 11.7 Job Payload Security

**How to Explain:** "Here's something I've seen teams completely miss until a security audit catches it. The job payload is JSONB in PostgreSQL. That payload often contains:

```
{ "user_email": "...", "api_key": "sk_live_xxxx", "ssn": "123-45-6789" }
```

Anyone with SELECT access to the jobs table — including DBAs, data analysts with read replicas, anyone who dumps the database for debugging — sees plaintext PII and credentials. This is an OWASP A02 Cryptographic Failures violation.

Here's the correct solution using envelope encryption:"

ENVELOPE ENCRYPTION WITH AWS KMS

WHY "ENVELOPE"? Never encrypt large data with KMS directly.

KMS has a 4KB limit and network latency. Instead:

Generate a local DEK, encrypt data with DEK, encrypt DEK with KMS.

JOB SUBMISSION (write path):

```

1. Generate DEK = random 256-bit key (in memory only)
2. payload_enc = AES-256-GCM.encrypt(payload, DEK)
3. dek_enc = KMS.encrypt(DEK, key_id='cmk-jobs')
4. INSERT INTO jobs:
 payload = payload_enc (ciphertext)
 encrypted_dek = dek_enc (wrapped key)
 kms_key_id = 'cmk-jobs'
→ PostgreSQL never sees plaintext

```

#### JOB EXECUTION (read path):

```

1. Worker fetches job row with payload_enc + encrypted_dek
2. DEK = KMS.decrypt(encrypted_dek) (1 network call)
3. payload = AES-256-GCM.decrypt(payload_enc, DEK)
4. DEK discarded from memory after use
5. CloudTrail logs: "worker-42 decrypted job-xyz at 14:22"

```

#### LIGHTER ALTERNATIVE (for most teams):

Store sensitive data in AWS Secrets Manager.

Payload contains only: { "secret\_ref": "arn:aws:secretsmanager:..." }

Worker fetches the actual secret at execution time.

Secrets rotate independently of job records.

"The KMS approach gives you full audit trail out of the box. Every decrypt call is in CloudTrail: which worker, which job, which time. For a compliance audit — GDPR, PCI-DSS, HIPAA — that log is exactly what you need to prove your system isn't leaking PII."

## 11.8 Kafka Consumer Rebalancing During Deploys

**How to Explain:** "This one causes mysterious latency spikes that are hard to diagnose. When we scale our worker fleet from 100 to 150 pods — either from auto-scaling or a rolling deployment — Kafka detects new consumers joining the consumer group. This triggers a group rebalance.

During a rebalance, ALL consumers in the group stop processing messages simultaneously. The group coordinator reassigns partitions. This is what people call the 'stop-the-world' rebalance. How long? Seconds to minutes depending on group size and partition count.

Here's what that looks like in your monitoring — a cliff in job throughput followed by recovery:"

#### REBALANCE IMPACT

```

Jobs/sec 1000 | ██████████
 500 | ██████████ ← rebalance gap
 0 | ██████████

```

time  
↑ worker added

### SOLUTION 1: Cooperative Sticky Rebalancing (Kafka 2.4+)

Config: `partition.assignment.strategy=CooperativeStickyAssignor`

Old behavior: STOP everyone → reassign all partitions → resume

New behavior: only move the partitions that need to change  
all other partitions keep consuming during move

Result: rebalance is incremental, not stop-the-world

### SOLUTION 2: Static Group Membership

Each worker pod: `group.instance.id = "worker-pod-7"`  
(set to the K8s pod name – stable across restarts)

On rolling deploy: pod restarts → same instance.id rejoins  
Kafka waits `session.timeout.ms` (45s) before rebalancing  
vs immediately triggering rebalance on disconnect

Effect: one pod at a time rebalances, others unaffected

### SOLUTION 3: Over-Provision Partitions (simplest)

Max workers = 500 → create 1,000 partitions

Scale 100→150 workers: new workers claim idle partitions  
No partition needs to move → no rebalance triggered

Cost: idle partitions consume minimal memory on brokers  
Benefit: zero rebalance impact on scale-out events

"In production I'd combine all three. CooperativeStickyAssignor as the base, static membership for rolling deploys, and over-provisioned partitions so scale-out never triggers rebalance at all."

## 11.9 SLO Burn Rate Alerting

**How to Explain:** "Let me tell you why the simple alerting we have in Section 8 — 'alert if p99 > 5 seconds' — causes alert fatigue at scale and gets ignored. The problem is it fires on brief, self-recovering spikes. A 30-second network hiccup at 2 AM triggers a page. On-call engineer wakes up, checks, it's already resolved. This happens 20 times a month. Eventually the team starts ignoring alerts. That's how real incidents get missed.

The fix is SLO-based burn rate alerting — the approach Google uses internally. Let me walk through it:"

SLO BURN RATE ALERTING

Our SLO: 99.9% of jobs start within 1 second of scheduled time  
Error budget: 0.1% per month = 43.8 minutes of allowed violations

BURN RATE = how fast you're consuming the error budget  
1x burn rate → budget exhausted in exactly 30 days (normal)  
6x burn rate → budget exhausted in 5 days (investigate)  
14x burn rate → budget exhausted in 2 hours (page NOW)

MULTI-WINDOW ALERTING (eliminates noise):

| Short window | Long window | Burn Rate | Severity                |
|--------------|-------------|-----------|-------------------------|
| 1 hour       | 5 minutes   | > 14.4x   | <span>🔴</span> PAGE NOW |
| 6 hours      | 30 minutes  | > 6x      | <span>🟡</span> Ticket   |
| 3 days       | 6 hours     | > 1x      | <span>🟢</span> Standup  |

TWO WINDOWS prevent both false positives and missed incidents:  
Short window alone: fires on brief spikes (false positive)  
Long window alone: fires too late on slow-burn incidents  
Both required: spike must be sustained to trigger alert

Prometheus rule for CRITICAL:  
(rate(job\_late\_starts[1h]) / rate(job\_starts[1h])) > 0.00144  
AND  
(rate(job\_late\_starts[5m]) / rate(job\_starts[5m])) > 0.00144

"The key insight is you're not alerting on instantaneous metric values — you're alerting on the rate at which you're consuming your reliability budget. A 30-second spike at 2 AM consumes 0.1% of the monthly budget — not worth waking anyone up. A sustained degradation consuming the budget 14x faster? Page immediately. Fewer pages, all of them actionable."

11.10 CAP Theorem — Explicit Trade-offs

**How to Explain:** "Interviewers love asking 'where does your system sit on the CAP spectrum?' The honest answer is: different components make different trade-offs, and I chose each one deliberately. Let me walk through each layer:"

CAP ANALYSIS BY COMPONENT

| Component | Consistency | Availability | Partition |
|-----------|-------------|--------------|-----------|
|           |             |              |           |



|                           |                                               |                                             |          |
|---------------------------|-----------------------------------------------|---------------------------------------------|----------|
| PostgreSQL<br>(job state) | ✓ Strong<br>ACID                              | Degraded if<br>primary down                 | ✓ Always |
| Redis<br>(schedule index) | ✗ Eventual<br>Rebuilt on<br>restart           | ✓ Always fast<br>reads, even<br>with stale  | ✓ Always |
| Kafka<br>(dispatch queue) | ✗ At-least<br>-once<br>duplicates<br>possible | ✓ High<br>throughput,<br>buffered<br>writes | ✓ Always |

OVERALL: AP system during partition → prioritize availability  
Consistency restored when partition heals

#### PARTITION SCENARIOS:

##### Redis unreachable:

- Scheduler falls back to PostgreSQL polling (Section 9.2)
- Jobs continue running, slightly higher DB load
- AP: available but with degraded performance

##### PostgreSQL unreachable:

- Reject new job submissions (can't guarantee durability)
- In-flight jobs continue (Redis + Kafka still work)
- CA: consistent for existing work, unavailable for new work

##### Kafka unreachable:

- Scheduler buffers due jobs in Redis in-flight set
- Retry Kafka publish every 5 seconds
- Workers drain existing in-flight work

"The hardest question interviewers ask is: 'Can a job ever be silently lost?' My answer is no, and here's the proof:

First, we always write to PostgreSQL before we write to Redis or Kafka. So the job is durable before it enters the fast path. Second, on any recovery — Redis rebuild, Kafka reconnect, scheduler restart — the first thing we do is scan PostgreSQL for any PENDING jobs older than 2 minutes and re-enqueue them. That's the reconciliation loop. The only failure mode where a job could be lost is if the PostgreSQL write itself fails — in which case the client receives an HTTP 500 and was never told the job was accepted. From the system's perspective, the job was never created. No silent loss."

---

*End of Job Scheduler Complete Interview Guide*